



Solana Keychain

Security Assessment

Prepared for
Solana Foundation

Prepared by
Otter Audits, LLC

Prepared on
September 30th, 2026

James Wang
Samuel Bétrisey
Gabriel Ottoboni
Biao Yu
Reyn Shaw
Eshan Ramesh

james.wang@osec.io
sam@osec.io
ottoboni@osec.io
ichild@osec.io
reyn@osec.io
ersh@osec.io

Table of Contents

1. Executive Summary		3
1.1. Overview		3
1.2. Key Findings		3
2. Scope		4
3. Findings		5
3.1. Severity by component		5
3.2. Findings summary		5
4. By Language		6
5. Advisories		9
OS-SKC-ADV-00	● Non-ASCII Signer Secret Can Panic Parsing	11
OS-SKC-ADV-01	● Duplicate Fireblocks Transactions Misclassified as Rejected	12
OS-SKC-ADV-02	● Shared Buffers Permit Signing Data Mutation	13
OS-SKC-ADV-03	● Double JSON Serialization Invalidates Wallet Authentication	14
OS-SKC-ADV-04	● Inconsistent Base URL Trailing-Slash Normalization	15
OS-SKC-ADV-05	● Configured Public Key Not Provider-Validated	17
OS-SKC-ADV-06	● Fordefi Returns Noncanonical Broadcast Transaction Signature	18
OS-SKC-ADV-07	● Race Condition Due To Shared Pending Transaction ID	19
OS-SKC-ADV-08	● Crossmint Returns Unverified Transaction Identifier	20
OS-SKC-ADV-09	● Broadcast Error Leaks Provider Transaction ID	21
OS-SKC-ADV-10	● SharedArrayBuffer Offsets Corrupt Signing Inputs	22
OS-SKC-ADV-11	● Credential Parsing Errors May Expose Secrets	23
OS-SKC-ADV-12	● Cancelled Transactions May Still Be Broadcast	24
OS-SKC-ADV-13	● Batch Abort Errors Omit Completed Signatures	25
OS-SKC-ADV-14	● UTF-8 BOM Removal Breaks Signature Verification	27
OS-SKC-ADV-15	● Mutable Designated Signers Stored by Reference	29
OS-SKC-ADV-16	● Whitespace Bypasses Signer Secret Length Validation	30
OS-SKC-ADV-17	● Fireblocks Accepts Invalid Polling Configuration Values	31
OS-SKC-ADV-18	● Ed25519 Verification Violates Solana Consensus Rules	32
6. Suggestions		33
OS-SKC-SUG-00	● Redundant Crossmint Signature Extraction Branching	34
OS-SKC-SUG-01	● Missing Crossmint API Key Structure Validation	35

<u>OS-SKC-SUG-02</u>	● CDP Signer Cannot Sign ALT Transactions	36
<u>OS-SKC-SUG-03</u>	● DFNS Signer Divergence From Reference Implementation	37
<u>OS-SKC-SUG-04</u>	● Pre-Send Failures Misclassified As Unconfirmed Broadcasts	39
<u>OS-SKC-SUG-05</u>	● Non-Broadcast Failures Misclassified as Broadcast-Unconfirmed	40
<u>OS-SKC-SUG-06</u>	● Imprecise Wallet Availability Check	42
<u>OS-SKC-SUG-07</u>	● Code Quality	43
A. Vulnerability Rating Scale		44
B. Procedure		45

1. Executive Summary

1.1. Overview

Solana Foundation engaged OtterSec to conduct a security assessment of the Rust, TypeScript, Python, and Go modules within Solana Keychain. This assessment was conducted between September 8th and September 22nd, 2026. For information on our auditing methodology, refer to [Appendix B](#).

1.2. Key Findings

We produced 27 findings throughout this audit engagement.

Among the most impactful findings, we identified that the Rust Crossmint backend checks only the byte length of the signer secret before slicing it at fixed offsets, so a secret containing multibyte characters panics during signer construction instead of returning a configuration error (● [OS-SKC-ADV-00](#)). We also found that the Rust Fireblocks backend treats every non-408 4xx response as proof that no transaction was accepted, even though Fireblocks answers a reused externalTxId with HTTP 400, so a caller may retry under a fresh identifier and create a duplicate transaction (● [OS-SKC-ADV-01](#)).

In the TypeScript module, `normalizeMessageBytes` relies on `.slice()`, which on Node.js buffers returns a shared-memory view rather than a copy, allowing a caller to mutate the bytes while asynchronous signing is pending and still pass local verification (● [OS-SKC-ADV-02](#)). Credential-decoding failures in the Crossmint, Memory, CDP, Openfort and Privy signers either escape as raw exceptions or are retained in the error's cause, so logging or serializing a `SignerError` may expose private keys and API secrets (● [OS-SKC-ADV-11](#)). In the Python module, the CDP and Openfort backends compute the wallet-authentication hash over one JSON serialization but let `httpx` re-serialize the body, so under supported `httpx` 0.27 versions the transmitted bytes differ and valid signing requests are rejected (● [OS-SKC-ADV-03](#)).

Finally, several backends across all four modules accept a configured public key without binding it to the remote key during initialization, so a misconfiguration surfaces only after signing and, for sending backends, potentially after broadcast (● [OS-SKC-ADV-05](#)).

Among the suggestions, the CDP signer omits the `network` field that CDP requires to resolve Address Lookup Tables, so versioned transactions depending on them are rejected across all four modules (● [OS-SKC-SUG-02](#)). The TypeScript DFNS signer left-pads signature components that DFNS already returns as fixed 32-byte values, which can turn a malformed response into a different signature rather than a rejected one, and interpolates `keyId` into the request path without percent-encoding (● [OS-SKC-SUG-03](#)). The Crossmint signer derives its seed from an API key without validating the expected three-part structure, so a malformed key may derive a seed under an unintended context (● [OS-SKC-SUG-01](#)).

2. Scope

The Rust module provides a unified, feature-gated interface for signing Solana messages and transactions through local keys, KMS services, MPC custodians, and managed-wallet providers. It separates sign-only, transaction-rewriting, and provider-broadcasting backends while verifying returned signatures and handling remote API, polling, and uncertain-broadcast errors.

The TypeScript module provides a unified interface for signing Solana transactions and messages through local keys, cloud KMS services, custody platforms, and remote wallet providers. It standardizes signer capabilities, error handling, transaction submission, and Solana Kit integration while keeping each backend independently tree-shakable.

The Python module provides a unified asynchronous interface for signing Solana messages and transactions through local keypairs, cloud KMS services, and managed custody providers. It standardizes transaction serialization, capability-specific signing and broadcasting, provider authentication, signature verification, and error handling across 14 backends. It also protects remote operations through HTTPS enforcement, response limits, redacted errors, and recovery metadata for ambiguous broadcasts.

The Go module provides a unified, security-focused interface for signing Solana messages and transactions through local keys, cloud KMS services, and managed custody providers. It standardizes transaction serialization, signature verification, error redaction, batching, and broadcasting while keeping each backend in a separate module to minimize dependencies.

The source code was delivered to us in a Git repository at <https://github.com/solana-foundation/solana-keychain>. This audit was performed against commit [e685e35](#). We reviewed remediation patches through commit [8ea75de](#). The `rust/src/ledger` directory and Ledger-related changes were excluded from the scope of this assessment.

3. Findings

Findings were rated using the criteria in [Appendix A](#).

We split the findings into **advisories** and **suggestions**. Advisories have an immediate impact and should be remediated as soon as possible. Suggestions do not have an immediate impact but will aid in mitigating future vulnerabilities.

Items are tracked with a stable identifier: **OS-SKC-ADV-*** for advisories and **OS-SKC-SUG-*** for suggestions.

3.1. Severity by component

Component	● Critical	● High	● Medium	● Low	● Info	Total
Rust	—	—	2	6	2	10
Typescript	—	—	2	8	4	14
Python	—	—	2	6	2	10
Go	—	—	1	4	5	10
Total	0	0	4	15	8	27

Findings affecting several components are counted once per component row; the Total row counts each finding once.

3.2. Findings summary

Severity	Count	
● Critical	0	
● High	0	
● Medium	4	
● Low	15	
● Informational	8	

4. By Language

Rust

ID	Severity	Status	Description
OS-SKC-ADV-00	● Medium	✓ Resolved	Non-ASCII Signer Secret Can Panic Parsing
OS-SKC-ADV-01	● Medium	✓ Resolved	Duplicate Fireblocks Transactions Misclassified as Rejected
OS-SKC-ADV-04	● Low	✓ Resolved	Inconsistent Base URL Trailing-Slash Normalization
OS-SKC-ADV-05	● Low	✓ Resolved	Configured Public Key Not Provider-Validated
OS-SKC-ADV-06	● Low	✓ Resolved	Fordefi Returns Noncanonical Broadcast Transaction Signature
OS-SKC-ADV-07	● Low	✓ Resolved	Race Condition Due To Shared Pending Transaction ID
OS-SKC-ADV-08	● Low	✓ Resolved	Crossmint Returns Unverified Transaction Identifier
OS-SKC-ADV-09	● Low	✓ Resolved	Broadcast Error Leaks Provider Transaction ID
OS-SKC-SUG-00	● Informational	✓ Resolved	Redundant Crossmint Signature Extraction Branching
OS-SKC-SUG-02	● Informational	✓ Resolved	CDP Signer Cannot Sign ALT Transactions

TypeScript

ID	Severity	Status	Description
OS-SKC-ADV-01	● Medium	✓ Resolved	Duplicate Fireblocks Transactions Misclassified as Rejected
OS-SKC-ADV-02	● Medium	✓ Resolved	Shared Buffers Permit Signing Data Mutation
OS-SKC-ADV-04	● Low	✓ Resolved	Inconsistent Base URL Trailing-Slash Normalization
OS-SKC-ADV-05	● Low	✓ Resolved	Configured Public Key Not Provider-Validated
OS-SKC-ADV-10	● Low	✓ Resolved	SharedArrayBuffer Offsets Corrupt Signing Inputs
OS-SKC-ADV-11	● Low	✓ Resolved	Credential Parsing Errors May Expose Secrets
OS-SKC-ADV-12	● Low	✓ Resolved	Cancelled Transactions May Still Be Broadcast
OS-SKC-ADV-13	● Low	✓ Resolved	Batch Abort Errors Omit Completed Signatures
OS-SKC-ADV-14	● Low	✓ Resolved	UTF-8 BOM Removal Breaks Signature Verification
OS-SKC-ADV-15	● Low	✓ Resolved	Mutable Designated Signers Stored by Reference

ID	Severity	Status	Description
OS-SKC-SUG-01	● Informational	✓ Resolved	Missing Crossmint API Key Structure Validation
OS-SKC-SUG-02	● Informational	✓ Resolved	CDP Signer Cannot Sign ALT Transactions
OS-SKC-SUG-03	● Informational	✓ Resolved	DFNS Signer Divergence From Reference Implementation
OS-SKC-SUG-04	● Informational	✓ Resolved	Pre-Send Failures Misclassified As Unconfirmed Broadcasts

Python

ID	Severity	Status	Description
OS-SKC-ADV-01	● Medium	✓ Resolved	Duplicate Fireblocks Transactions Misclassified as Rejected
OS-SKC-ADV-03	● Medium	✓ Resolved	Double JSON Serialization Invalidates Wallet Authentication
OS-SKC-ADV-05	● Low	✓ Resolved	Configured Public Key Not Provider-Validated
OS-SKC-ADV-06	● Low	✓ Resolved	Fordefi Returns Noncanonical Broadcast Transaction Signature
OS-SKC-ADV-07	● Low	✓ Resolved	Race Condition Due To Shared Pending Transaction ID
OS-SKC-ADV-15	● Low	✓ Resolved	Mutable Designated Signers Stored by Reference
OS-SKC-ADV-16	● Low	✓ Resolved	Whitespace Bypasses Signer Secret Length Validation
OS-SKC-ADV-17	● Low	✓ Resolved	Fireblocks Accepts Invalid Polling Configuration Values
OS-SKC-SUG-00	● Informational	✓ Resolved	Redundant Crossmint Signature Extraction Branching
OS-SKC-SUG-02	● Informational	✓ Resolved	CDP Signer Cannot Sign ALT Transactions

Go

ID	Severity	Status	Description
OS-SKC-ADV-01	● Medium	✓ Resolved	Duplicate Fireblocks Transactions Misclassified as Rejected
OS-SKC-ADV-04	● Low	✓ Resolved	Inconsistent Base URL Trailing-Slash Normalization
OS-SKC-ADV-05	● Low	✓ Resolved	Configured Public Key Not Provider-Validated

ID	Severity	Status	Description
OS-SKC-ADV-15	● Low	✓ Resolved	Mutable Designated Signers Stored by Reference
OS-SKC-ADV-18	● Low	✓ Resolved	Ed25519 Verification Violates Solana Consensus Rules
OS-SKC-SUG-00	● Informational	✓ Resolved	Redundant Crossmint Signature Extraction Branching
OS-SKC-SUG-02	● Informational	✓ Resolved	CDP Signer Cannot Sign ALT Transactions
OS-SKC-SUG-05	● Informational	✓ Resolved	Non-Broadcast Failures Misclassified as Broadcast-Unconfirmed
OS-SKC-SUG-06	● Informational	✓ Resolved	Imprecise Wallet Availability Check
OS-SKC-SUG-07	● Informational	✓ Resolved	Code Quality

5. Advisories

Here, we present a technical analysis of the 19 vulnerabilities we identified during our audit. These vulnerabilities have **immediate** security implications, and we recommend remediation as soon as possible.

ID	Severity	Status	Description
OS-SKC-ADV-00	● Medium (Rust)	✓ Resolved	Non-ASCII Signer Secret Can Panic Parsing
OS-SKC-ADV-01	● Medium (Rust, Typescript, Python, Go)	✓ Resolved	Duplicate Fireblocks Transactions Misclassified as Rejected
OS-SKC-ADV-02	● Medium (Typescript)	✓ Resolved	Shared Buffers Permit Signing Data Mutation
OS-SKC-ADV-03	● Medium (Python)	✓ Resolved	Double JSON Serialization Invalidates Wallet Authentication
OS-SKC-ADV-04	● Low (Rust, Typescript, Go)	✓ Resolved	Inconsistent Base URL Trailing-Slash Normalization
OS-SKC-ADV-05	● Low (Rust, Typescript, Python, Go)	✓ Resolved	Configured Public Key Not Provider-Validated
OS-SKC-ADV-06	● Low (Rust, Python)	✓ Resolved	Fordefi Returns Noncanonical Broadcast Transaction Signature
OS-SKC-ADV-07	● Low (Rust, Python)	✓ Resolved	Race Condition Due To Shared Pending Transaction ID
OS-SKC-ADV-08	● Low (Rust)	✓ Resolved	Crossmint Returns Unverified Transaction Identifier
OS-SKC-ADV-09	● Low (Rust)	✓ Resolved	Broadcast Error Leaks Provider Transaction ID
OS-SKC-ADV-10	● Low (Typescript)	✓ Resolved	SharedArrayBuffer Offsets Corrupt Signing Inputs
OS-SKC-ADV-11	● Low (Typescript)	✓ Resolved	Credential Parsing Errors May Expose Secrets
OS-SKC-ADV-12	● Low (Typescript)	✓ Resolved	Cancelled Transactions May Still Be Broadcast
OS-SKC-ADV-13	● Low (Typescript)	✓ Resolved	Batch Abort Errors Omit Completed Signatures
OS-SKC-ADV-14	● Low (Typescript)	✓ Resolved	UTF-8 BOM Removal Breaks Signature Verification
OS-SKC-ADV-15	● Low (Python, Typescript, Go)	✓ Resolved	Mutable Designated Signers Stored by Reference

ID	Severity	Status	Description
<u>OS-SKC-ADV-16</u>	● Low (Python)	✓ Resolved	Whitespace Bypasses Signer Secret Length Validation
<u>OS-SKC-ADV-17</u>	● Low (Python)	✓ Resolved	Fireblocks Accepts Invalid Polling Configuration Values
<u>OS-SKC-ADV-18</u>	● Low (Go)	✓ Resolved	Ed25519 Verification Violates Solana Consensus Rules

Non-ASCII Signer Secret Can Panic Parsing

ID	Severity	Status
OS-SKC-ADV-00	● Medium	✓ Resolved

Description

`CrossmintSigner::derive_signing_key` validates only that `raw_secret` is 64 bytes long, not that it contains 64 ASCII hexadecimal characters. It then slices the UTF-8 string at fixed byte offsets (`&raw_secret[i..i + 2]`), which panics if an offset falls inside a multibyte character. A malformed configuration can therefore crash signer construction instead of returning the intended `ConfigError`, creating a local denial-of-service condition.

```
rust/src/crossmint/mod.rs
1  fn derive_signing_key(
2      secret: &str,
3      api_key: &str,
4  ) -> Result<ed25519_dalek::SigningKey, SignerError> {
5      use hkdf::Hkdf;
6      use sha2::Sha256
7      let (project_id, environment) = Self::parse_api_key(api_key)?
8      let raw_secret = secret.strip_prefix("xmsk1_").unwrap_or(secret);
9      if raw_secret.len() != 64 {
10         return Err(SignerError::ConfigError(format!(
11             "signer_secret must be a 64-char hex string (got {})",
12             raw_secret.len()
13         )));
14     }
15     [...]
16 }
```

Remediation

Decode the entire string with `hex::decode` or first require `raw_secret.is_ascii()` and `chars().all(|c| c.is_ascii_hexdigit())` before byte-indexed parsing.

Patch

Resolved in [PR#314](#).

Duplicate Fireblocks Transactions Misclassified as Rejected

ID	Severity	Status
OS-SKC-ADV-01	● Medium	✓ Resolved

Description

`unconfirmed_unless_rejected` assumes every non-408 4xx response proves that no relevant transaction was previously accepted. However, Fireblocks returns HTTP `400` with error code `1438` when the supplied `externalTxId` was already used, meaning an earlier transaction exists under that identifier. Returning the original error therefore hides the possibility that the prior transaction has progressed or already been broadcast. A caller may incorrectly treat the operation as safely rejected and retry with a new external ID, potentially creating a duplicate transaction. This issue is also present in the TypeScript, Python, and Go modules.

Remediation

Duplicate-ID responses should be handled separately by reconciling the existing transaction through its `externalTxId`, rather than treating them as definitive non-acceptance. Fireblocks error codes

Patch

Resolved in [PR#314](#).

Shared Buffers Permit Signing Data Mutation

ID	Severity	Status
OS-SKC-ADV-02	● Medium	✓ Resolved

Description

`normalizeMessageBytes()` attempts to isolate caller-supplied data by calling `.slice()` on `Uint8Array` inputs. However, `Node.js` `Buffer` extends `Uint8Array`, while `Buffer.slice()` returns a shared-memory view rather than an independent copy. If the caller modifies or reuses the underlying buffer while asynchronous signing is pending, the signer may process different bytes from those originally submitted. In AWS KMS, signature verification uses the same shared view, so a signature over the modified message can still pass local verification. This creates a time-of-check/time-of-use risk and breaks the function's guarantee that signing operates on a stable byte snapshot.

```
typescript/packages/core/src/utls.ts
```

```
1  /** Copies caller-supplied bytes into a fresh, zero-offset `Uint8Array`. */
2  export function normalizeMessageBytes(bytes: ArrayLike<number>): Uint8Array {
3      return bytes instanceof Uint8Array ? bytes.slice() : new
         Uint8Array(Array.from(bytes));
4  }
```

Remediation

Create an independent byte snapshot with `Uint8Array.from(bytes)` before starting asynchronous signing. Use that same immutable copy for both the signing request and subsequent signature verification.

Patch

Resolved in [PR#314](#).

Double JSON Serialization Invalidates Wallet Authentication

ID	Severity	Status
OS-SKC-ADV-03	● Medium	✓ Resolved

Description

CDP and Openfort compute the wallet-auth JWT over one JSON serialization but pass `json_body` to `httpx`, which independently serializes the same object for transmission. Under supported `httpx 0.27.x` versions, its JSON separators differ from Keychain's canonical serialization in `fetch_signer_json`, producing different request bytes.

```
python/src/solana_keychain/core/http.py
```

```

1  async def fetch_signer_json(
2      *,
3      url: str,
4      provider_name: str,
5      method: str = "GET",
6      headers: dict[str, str] | None = None,
7      json_body: Any | None = None,
8      content: bytes | None = None,
9      timeout_seconds: float = DEFAULT_REQUEST_TIMEOUT_SECONDS,
10     client: httpx.AsyncClient | None = None,
11 ) -> Any:
12     """Perform a remote signer API request and parse the JSON response, mapping
13     failures to the standard signer error pipeline:
14     [...]
15     ``json_body`` and ``content`` are mutually exclusive. Use ``content`` when the
16     request bytes must match a signature computed over them exactly (request-stamping
17     auth schemes) – re-serialization would change the bytes.
18     [...]
19     """

```

Because the JWT's `reqHash` authenticates the body, the provider calculates a different hash and rejects an otherwise valid signing request. The issue may appear absent under `httpx 0.28` because its compact separators happen to match, but correctness should not depend on the HTTP client version.

Remediation

Serialize the body once using the JWT's canonical format and pass those exact bytes through `content` instead of `json_body`.

Patch

Resolved in [PR#314](#).

Inconsistent Base URL Trailing-Slash Normalization

ID	Severity	Status
OS-SKC-ADV-04	● Low	✓ Resolved

Description

Only Fordefi and Crossmint normalize `base_url` with `trim_end_matches('/', '')`. Other backends may therefore construct paths containing `//` when given a trailing slash. Although some servers normalize these URLs, others may redirect, reject the request, or invalidate authentication that signs an exact path. This issue is also present in the TypeScript and Go modules.

vault/src/processor/vault_deposit.rs

```

1  pub fn process_vault_deposit(
2      _program_id: &Pubkey,
3      accounts: &[AccountInfo],
4      params: VaultDepositParams,
5  ) -> ProgramResult {
6      [...]
7      } else if total_base == 0 {
8          // Pool is quote-only; only quote-side deposits have a defined rate.
9          if base_lots_to_deposit > 0 {
10             return Err(VaultError::ImbalancedDeposit.into());
11         }
12         if quote_lots_to_deposit == 0 {
13             return Err(VaultError::ZeroDeposit.into());
14         }
15         let lp_u128: u128 = (lp_supply as u128)
16             .checked_mul(quote_lots_to_deposit as u128)
17             .ok_or(VaultError::ArithmeticOverflow)?
18             .checked_div(total_quote as u128)
19             .ok_or(VaultError::ArithmeticOverflow)?;
20         let lp = u64::try_from(lp_u128).map_err(|_| VaultError::ArithmeticOverflow)?;
21         if lp == 0 {
22             return Err(VaultError::ZeroLpMinted.into());
23         }
24         (0u64, quote_lots_to_deposit, lp)
25     }
26     [...]
27 }

```

Remediation

Ensure all backends apply one shared URL-normalization/joining strategy and test equivalent inputs with and without a trailing slash.

Patch

Resolved in [PR#314](#).

Configured Public Key Not Provider-Validated

ID	Severity	Status
OS-SKC-ADV-05	● Low	✓ Resolved

Description

Several backends trust a user-supplied `public_key` without binding it to the configured remote key, vault, or account during initialization. A mismatch is detected only after signing when `verify_or_reject` fails, which is operationally safe for sign-only backends but too late for a `SendingSigner` that may already have broadcast. Fordefi native auto validates that the configured key is the submitted transaction's sole signer, but this does not independently prove that the Fordefi vault controls that key.

Initialization or availability checks should fetch and compare the provider's public key, or verify a non-broadcast challenge signature, before accepting transactions. This invariant should be documented and enforced for `aws_kms`, `fordefi`, `gcp_kms`, `turnkey`, and `vault`, to prevent future sending backends from discovering misconfiguration post-broadcast. This issue is also present in the TypeScript, Python and Go modules.

Remediation

During initialization, fetch the provider's public key or verify a non-broadcast challenge signature and ensure it matches the configured `public_key`.

Patch

Resolved in [PR#314](#).

Fordefi Returns Noncanonical Broadcast Transaction Signature

ID	Severity	Status
OS-SKC-ADV-06	● Low	✓ Resolved

Description

`SendingSigner::sign_and_send_transaction` must return the broadcast transaction's slot-0 fee-payer signature, which Solana uses as its transaction ID. `finish_broadcast` instead returns the configured vault's verified signature from its required-signer position in Fordefi's rewritten transaction. If Fordefi changes the fee payer, that signature may not be slot zero, making the result unusable for standard RPC lookup, confirmation, and reconciliation. This issue is also present in the Python module.

```
rust/src/fordefi/mod.rs
```

```

1  async fn finish_broadcast(&self, tx_id: &str) -> Result<Signature, SignerError> {
2      let result = self.core.poll_for_result(tx_id, true).await?;
3      let wire_bytes = FordefiCore::decode_raw_transaction(&result)?;
4      let (_, signature) =
5          extract_and_verify_rewritten_transaction(&wire_bytes, &self.core.public_key)?;
6      Ok(signature)
7  }
```

Remediation

After verifying the vault signature, the function should extract and return the rewritten transaction's nondefault slot-0 signature, preferably verifying it against the returned fee payer and message.

Patch

Resolved in [PR#314](#).

Race Condition Due To Shared Pending Transaction ID

ID	Severity	Status
OS-SKC-ADV-07	● Low	✓ Resolved

Description

`PendingTransactionId` stores only one provider transaction ID behind a shared mutex, so it cannot safely track multiple concurrent `sign_and_send` operations. If two Crossmint or Fordefi native-auto calls overlap, the later call can overwrite the earlier call's ID, while either call may subsequently clear the shared slot. Consequently, cancellation recovery may return the wrong transaction ID or no ID at all, preventing reliable reconciliation of an accepted or broadcast transaction. This issue is also present in the Python module.

Remediation

Ensure the state is scoped per operation or keyed by a unique request identifier.

Patch

Resolved in [PR#314](#).

Crossmint Returns Unverified Transaction Identifier

ID	Severity	Status
OS-SKC-ADV-08	● Low	✓ Resolved

Description

When `onChain.transaction` is present but decoding or signature verification fails, `extract_signature_from_response` suppresses that error whenever `onChain.txId` is also available. It then accepts `txId` after only base58 decoding and a 64-byte length check, without proving that it identifies the returned transaction or corresponds to the expected signer and message. Consequently, a malformed or inconsistent Crossmint response can produce a successful result despite failed cryptographic validation. This also diverges from the TypeScript implementation, which treats verification failure as terminal.

Remediation

Treat returned-transaction validation failures as terminal, allowing `onChain.txId` fallback only when no serialized transaction is provided.

Patch

Resolved in [PR#314](#).

Broadcast Error Leaks Provider Transaction ID

ID	Severity	Status
OS-SKC-ADV-09	● Low	✓ Resolved

Description

BroadcastUnconfirmed interpolates `provider_tx_id` into its `Display` output, so ordinary error logging can expose an internal provider-side identifier. This weakens the crate's redaction guarantees and differs from other language implementations that keep recovery metadata programmatically accessible but out of formatted messages.

```
rust/src/error.rs
```

```

1  #[derive(Error)]
2  pub enum SignerError {
3      [...]
4      #[error("Broadcast unconfirmed; the provider may have executed the transaction
5         (provider transaction id: {})", provider_tx_id.as_deref().unwrap_or("unknown"))]
6         BroadcastUnconfirmed {
7             provider_tx_id: Option<String>,
8             provider_status: Option<u16>,
9             idempotency_key: Option<String>,
10             transaction_signature: Option<Box<Signature>>,
11             detail: String,
12         },
13     [...]
14 }
```

Remediation

Utilize a generic display message to ensure consistency with other implementations.

Patch

Resolved in [PR#314](#).

SharedArrayBuffer Offsets Corrupt Signing Inputs

ID	Severity	Status
OS-SKC-ADV-10	● Low	✓ Resolved

Description

`solana-keychain` permits Solana Kit 8.0.0 through its peer ranges and also uses the affected modules as development dependencies. This version mishandles `SharedArrayBuffer`-backed views with non-zero offsets by applying the view offset again after copying its bytes. Consequently, Keychain signing or verification paths may operate on a truncated suffix of the intended input, or even on empty bytes. A caller could therefore receive a valid signature that does not authenticate the complete intended message or transaction. Likewise, signatures returned by remote providers may be incorrectly accepted or rejected because verification processes the wrong bytes.

Remediation

Raise the affected `@solana/*` peer and development dependencies to version `>=8.1.0`. Additionally, copy inputs into an owned, zero-offset `Uint8Array` and use that same snapshot for signing, submission, and verification.

Patch

Resolved in [PR#314](#).

Credential Parsing Errors May Expose Secrets

ID	Severity	Status
OS-SKC-ADV-11	● Low	✓ Resolved

Description

Credential-decoding libraries may include the rejected input in their exception messages, causing malformed secrets to leak through error handling. Crossmint's `deriveSignerSeed` performs `base16` and `base58` decoding outside any try-catch, allowing raw exceptions containing the credential to escape directly. Memory, CDP, Openfort, and Privy catch equivalent parsing failures but retain the original exception in `context.cause`.

Replacing only the top-level error message is therefore insufficient because logging, inspecting, or serializing the complete `SignerError` may expose the underlying cause. Affected values include Solana private keys, CDP API keys, P-256 wallet secrets, Privy authorization keys, and Crossmint signer seeds as listed below:

- Memory: `parseBase58PrivateKey` preserves the `base58` decoder error as cause, potentially exposing the supplied Solana private key through logs or serialization.
- CDP: `loadApiKey` and `loadWalletKey` retain decoding or key-import exceptions that may reveal CDP API and wallet secrets.
- Openfort: `loadWalletKey` attaches raw PEM/`base64` decoding or `PKCS#8` import errors, potentially leaking the wallet-authorization key.
- Privy: `generatePrivyAuthorizationSignature` preserves errors from `parseP256PrivateKey`, which may expose the supplied Privy authorization key.

Disclosure of this material can grant unauthorized signing access and remains dangerous even when initialization itself fails.

Remediation

Catch credential-decoding and key-parsing failures at their source and return only generic, sanitized errors. Do not retain raw exceptions in `cause` or other error context when they may contain secret key material.

Patch

Resolved in [PR#314](#).

Cancelled Transactions May Still Be Broadcast

ID	Severity	Status
OS-SKC-ADV-12	● Low	✓ Resolved

Description

`send::signAndSendTransaction` forwards `abortSignal` but never checks whether it has already been aborted. If cancellation occurs during signing, a `sendTransaction` callback that ignores the signal may still broadcast the completed transaction. This violates the caller's cancellation intent and can cause an unwanted on-chain submission after the operation appears cancelled.

typescript/packages/core/src/send.ts

```

1  export async function signAndSendTransaction<TAddress extends string>(
2    signer: SolanaSigner<TAddress>,
3    transaction: SignableTransaction,
4    config?: SignAndSendTransactionConfig,
5  ): Promise<SignatureBytes> {
6    const abortSignal = config?.abortSignal;
7    [...]
8    let signature: SignatureBytes | void;
9    try {
10     signature = await sendTransaction(signedTransaction, { abortSignal });
11   } catch (cause) {
12     throw createSignerError(
13       SignerErrorCode.BROADCAST_UNCONFIRMED,
14       {
15         address: signer.address,
16         message: 'The transaction may have been broadcast; reconcile its
17           signature before retrying',
18         transactionSignature: feePayerSignature,
19       },
20       cause,
21     );
22   }
23   [...]
24 }
```

Remediation

Call `abortSignal?.throwIfAborted()` at function entry and again immediately before invoking the sender. Keep the second check outside the broadcast `try` block so pre-broadcast cancellation is not misreported as `BROADCAST_UNCONFIRMED`.

Patch

Resolved in [PR#314](#).

Batch Abort Errors Omit Completed Signatures

ID	Severity	Status
OS-SKC-ADV-13	● Low	✓ Resolved

Description

`batch::signBatchSequential` attaches completed results only when `fn` throws a `SignerError`, while cancellation and other exceptions propagate without partial-progress context. If a batch is aborted after earlier items succeed, callers cannot determine which signatures or provider-side operations were already completed. Retrying the entire batch may therefore repeat completed work and, for stateful signing or submission backends, risk duplicate transaction processing.

```
typescript/packages/core/src/batch.ts
```

```

1  export async function signBatchSequential<TItem, TResult>(
2    items: readonly TItem[],
3    fn: (item: TItem, index: number) => Promise<TResult>,
4    delayMs: number,
5    completedKey: string,
6    abortSignal?: AbortSignal,
7  ): Promise<readonly TResult[]> {
8    abortSignal?.throwIfAborted();
9    const results: TResult[] = [];
10   for (const [index, item] of items.entries()) {
11     if (delayMs > 0 && index > 0) {
12       await abortableDelay(delayMs, abortSignal);
13     }
14     abortSignal?.throwIfAborted();
15     try {
16       results.push(await fn(item, index));
17     } catch (error) {
18       if (!(error instanceof SignerError)) {
19         throw error;
20       }
21       throwSignerError(error.code, {
22         ...error.context,
23         [completedKey]: [...results],
24         failedIndex: index,
25       });
26     }
27   }
28   return results;
29 }
```

Remediation

Wrap abort and non-signer failures in a structured error that includes `failedIndex` and the results accumulated under `completedKey`.

Patch

Resolved in [PR#314](#).

UTF-8 BOM Removal Breaks Signature Verification

ID	Severity	Status
OS-SKC-ADV-14	● Low	✓ Resolved

Description

In `cdp-signer`, `decodeMessageBytes` uses the default `TextDecoder` BOM behavior, which consumes a leading UTF-8 BOM (`EF BB BF`) instead of including it in the decoded string. CDP consequently receives and signs the message content without those three leading bytes.

```
typescript/packages/core/src/batch.ts
```

```
1 private decodeMessageBytes(messageBytes: Uint8Array): string {
2     try {
3         utf8Decoder ||= new TextDecoder('utf-8', { fatal: true });
4         return utf8Decoder.decode(messageBytes);
5     }[...]
6 }
```

However, `signMessages` verifies the returned signature against the original `message.content`, which still contains the BOM. The signed and verified byte sequences therefore differ, causing valid BOM-prefixed messages to fail with `SIGNER_SIGNING_FAILED`.

```
typescript/packages/core/src/batch.ts
```

```
1 async signMessages(
2     messages: readonly SignableMessage[],
3     config?: MessagePartialSignerConfig,
4 ): Promise<readonly SignatureDictionary[]> {
5     return await signBatchStaggered(
6         messages,
7         async message => {
8             const utf8Message = this.decodeMessageBytes(message.content);
9             const signatureBytes = await this.callSignMessage(utf8Message,
10                 config?.abortSignal);
11             await assertSignatureValid({
12                 data: message.content,
13                 signature: signatureBytes,
14                 signerAddress: this.address,
15             });
16             [...];
17         }, [...];
18     );
19 }
```

Remediation

Construct the decoder with `{ fatal: true, ignoreBOM: true }` so the BOM is preserved in the string sent to CDP.

Patch

Resolved in [PR#314](#).

Mutable Designated Signers Stored by Reference

ID	Severity	Status
OS-SKC-ADV-15	● Low	✓ Resolved

Description

`UtilaSigner::__init__` assigns `config.designated_signers` directly, retaining an alias to the caller-owned mutable list. If the caller later modifies that list, the signer's future `transactions: initiate` requests silently use the updated signer identities without reconfiguration or validation. This violates the expectation that constructor configuration is snapshotted and could unintentionally add or remove authorized signers. This issue is also present in the TypeScript and Go modules.

```
python/src/solana_keychain/utila/signer.py
1  def __init__(self, config: UtilaSignerConfig) -> None:
2      [...]
3      self._designated_signers = (
4          config.designated_signers
5          if config.designated_signers is not None
6          else [f"users/{config.service_account_email}"]
7      )
8      self._http_client = config.http_client
9      self._public_key: Pubkey | None = None
```

Remediation

Store a defensive copy, such as `list(config.designated_signers)`, when initializing the signer.

Patch

Resolved in [PR#314](#).

Whitespace Bypasses Signer Secret Length Validation

ID	Severity	Status
OS-SKC-ADV-16	● Low	✓ Resolved

Description

`signer::derive_signing_key` validates the string length before calling `bytes.fromhex`, but `fromhex` silently ignores ASCII whitespace between byte pairs. A 64-character input containing whitespace can therefore pass the initial check while decoding to fewer than the required 32 bytes. HKDF will still accept this shorter input and derive a valid but unintended keypair from a malformed signer secret.

```
python/src/solana_keychain/utila/signer.py
```

```

1  def derive_signing_key(secret: str, api_key: str) -> Keypair:
2      [...]
3      raw_secret = secret.removeprefix("xmsk1_")
4      if len(raw_secret) != SIGNER_SECRET_HEX_LENGTH:
5          raise SignerError(
6              SignerErrorCode.CONFIG_ERROR,
7              f"signer_secret must be a {SIGNER_SECRET_HEX_LENGTH}-char hex string "
8              f"(got {len(raw_secret)})",
9          )
10     try:
11         ikm = bytes.fromhex(raw_secret)
12     except ValueError:
13         raise SignerError(SignerErrorCode.CONFIG_ERROR, "signer_secret is not valid
14             hex") from None
15
16     hkdf = HKDF(
17         algorithm=hashes.SHA256(),
18         length=32,
19         salt=b"crossmint",
20         info=f"{project_id}:{environment}:solana-ed25519".encode(),
21     )
22     return Keypair.from_seed(hkdf.derive(ikm))

```

Remediation

After hexadecimal decoding, require `len(ikm) == 32` and reject non-canonical inputs containing whitespace.

Patch

Resolved in [PR#314](#).

Fireblocks Accepts Invalid Polling Configuration Values

ID	Severity	Status
OS-SKC-ADV-17	● Low	✓ Resolved

Description

`FireblocksSigner` accepts zero or negative values for `poll_interval_ms` and `max_poll_attempts` without validation. A non-positive attempt count can cause signing to time out immediately, while a non-positive interval may create a tight polling loop that unnecessarily consumes Fireblocks API capacity.

Remediation

Validate both fields are strictly positive during construction, before creating the signer.

Patch

Resolved in [PR#314](#).

Ed25519 Verification Violates Solana Consensus Rules

ID	Severity	Status
OS-SKC-ADV-18	● Low	✓ Resolved

Description

`VerifyEd25519` uses Go's `crypto/ed25519`, whose stricter verification rules are not ZIP-215 compliant and may reject signatures that Solana accepts under SIMD-0376 consensus rules.

```

go/core/verify.go
1 // VerifyEd25519 reports whether sig is a valid Ed25519 signature of message by
2 // pubkey. Remote and KMS backends call this to verify a signature returned by the
3 // service before surfacing it.
4 func VerifyEd25519(pubkey solana.PublicKey, message []byte, sig solana.Signature) bool
5 {
6     return ed25519.Verify(ed25519.PublicKey(pubkey.Bytes()), message, sig[:])
7 }

```

Remediation

Use `hdevalence/ed25519consensus` to align off-chain verification with on-chain Ed25519 semantics and avoid false rejection of valid signatures.

Patch

Resolved in [PR#314](#).

6. Suggestions

Here, we present a discussion of the 8 suggestions we identified during our audit. While these findings do not present an immediate security impact, they represent anti-patterns and may result in security issues in the future.

ID	Severity	Status	Description
OS-SKC-SUG-00	● Informational (Rust, Python, Go)	✓ Resolved	Redundant Crossmint Signature Extraction Branching
OS-SKC-SUG-01	● Informational (Typescript)	✓ Resolved	Missing Crossmint API Key Structure Validation
OS-SKC-SUG-02	● Informational (Go, Rust, Typescript, Python)	✓ Resolved	CDP Signer Cannot Sign ALT Transactions
OS-SKC-SUG-03	● Informational (Typescript)	✓ Resolved	DFNS Signer Divergence From Reference Implementation
OS-SKC-SUG-04	● Informational (Typescript)	✓ Resolved	Pre-Send Failures Misclassified As Unconfirmed Broadcasts
OS-SKC-SUG-05	● Informational (Go)	✓ Resolved	Non-Broadcast Failures Misclassified as Broadcast-Unconfirmed
OS-SKC-SUG-06	● Informational (Go)	✓ Resolved	Imprecise Wallet Availability Check
OS-SKC-SUG-07	● Informational (Go)	✓ Resolved	Code Quality

Redundant Crossmint Signature Extraction Branching

ID	Severity	Status
OS-SKC-SUG-00	● Informational	✓ Resolved

Description

In `CrossmintSigner::extract_signature_from_response`, after `extract_signature_from_serialized_transaction` succeeds, `signature` is already the transaction's nondefault slot-0 fee-payer signature and has been cryptographically verified. `broadcast_transaction_id(&returned)` repeats the same account/signature presence checks and returns that identical slot-0 signature. Because both branches produce the same value, comparing `returned.message.serialize()` with `expected_message` has no behavioral or security effect.

rust/src/crossmint/mod.rs

```

1  fn extract_signature_from_response(
2      &self,
3      response: &TransactionResponse,
4      expected_message: &[u8],
5  ) -> Result<Signature, SignerError> {
6      if let Some(on_chain) = &response.on_chain {
7          if let Some(serialized_transaction) = &on_chain.transaction {
8              match
9                  self.extract_signature_from_serialized_transaction(serialized_transaction)
10             {
11                 Ok((signature, returned)) => {
12                     if returned.message.serialize() == expected_message {
13                         return Ok(signature);
14                     }
15                     return Self::broadcast_transaction_id(&returned);
16                 }
17                 [...]
18             }
19         }
20         [...]
21     }

```

Remediation

Remove the redundant validation logic from Rust, Python, and Go implementations.

Patch

Resolved in [PR#314](#) and [PR#314](#).

Missing Crossmint API Key Structure Validation

ID	Severity	Status
OS-SKC-SUG-01	● Informational	✓ Resolved

Description

`deriveSignerSeed()` in `crossmint-signer` splits `apiKey` on `_` but assumes the expected `{ck|sk}_{environment}_{base58data}` structure without validating the resulting components. A malformed key can therefore produce missing or empty derivation inputs, trigger an unrelated decoder error, or derive a seed under an unintended `undefined` context.

```
typescript/packages/crossmint/src/crossmint-signer.ts
```

```

1  async function deriveSignerSeed(secret: string, apiKey: string): Promise<Uint8Array> {
2      const { hkdfSync } = await import('node:crypto');
3      const rawSecret = secret.startsWith('xmsk1_') ? secret.slice(6) : secret;
4      if (rawSecret.length !== 64) {
5          throwSignerError(SignerErrorCode.CONFIG_ERROR, {
6              message: `signerSecret must be a 64-char hex string, got
7                  ${rawSecret.length} chars`,
8              });
9      }
10     base16Encoder ||= getBase16Encoder();
11     const ikm = Buffer.from(base16Encoder.encode(rawSecret));
12
13     // Parse API key: {ck|sk}_{environment}_{base58data}
14     // base58-decoded data is UTF-8: "projectId:nacl_signature"
15     const parts = apiKey.split('_');
16     const environment = parts[1];
17     const base58Data = parts.slice(2).join('_');
18     base58Encoder ||= getBase58Encoder();
19     const decoded = base58Encoder.encode(base58Data);
20     const projectId = new TextDecoder().decode(decoded).split(':')[0];
21
22     const info = `${projectId}:${environment}:solana-ed25519`;
23     return new Uint8Array(hkdfSync('sha256', ikm, 'crossmint', info, 32));
24 }
```

Remediation

Validate that `apiKey` contains exactly three non-empty components with a supported prefix before decoding it or deriving the signer seed.

Patch

Resolved in [PR#314](#).

CDP Signer Cannot Sign ALT Transactions

ID	Severity	Status
OS-SKC-SUG-02	● Informational	✓ Resolved

Description

The CDP signer submits only the serialized transaction and omits the `network` required by CDP for transactions using Address Lookup Tables (ALTs). Without this value, CDP cannot determine the Solana cluster from which referenced lookup-table accounts must be resolved. Consequently, the server rejects otherwise valid versioned transactions that depend on ALTs, preventing them from being signed through this backend. This issue is also present in the Rust, Python and Go modules.

Remediation

Add a validated `network` field to `CdpSignerConfig` and include it in the sign-transaction request body and authenticated request hash. CI integration tests should cover ALT-backed transactions on supported networks to prevent regressions.

Patch

Resolved in [PR#314](#).

DFNS Signer Divergence From Reference Implementation

ID	Severity	Status
OS-SKC-SUG-03	● Informational	✓ Resolved

Description

1. The comment in `dfns-signer::padSignatureComponent` states DFNS may return components shorter than 32 bytes. However, on testing, DFNS returned Ed25519 `r` and `s` as fixed-width 32-byte components, preserving both leading and trailing zero bytes. The reference SDK also concatenates them without padding. This discrepancy may transform a malformed or truncated API response into a different 32-byte component, obscuring the actual response-format violation.

```
typescript/packages/dfns/src/dfns-signer.ts
```

```

1  /**
2   * Pad signature component to exactly 32 bytes.
3   * Components from Dfns may be shorter than 32 bytes and need left-padding with
4   * zeros.
5   */
6   function padSignatureComponent(hex: string): Uint8Array {
7       const bytes = hexToBytes(hex);
8       if (bytes.length > 32) {
9           throwSignerError(SignerErrorCode.SIGNING_FAILED, {
10               message: `Invalid signature component length: ${bytes.length} (max
11                   32)`,
12           });
13       }
14       const padded = new Uint8Array(32);
15       padded.set(bytes, 32 - bytes.length);
16       return padded;
17   }
```

2. `dfns-signer::sendSignatureRequest` inserts the server-issued `keyId` directly into `/keys/{keyId}/signatures` without percent-encoding it. Although the identifier is provider-controlled and presents no security issue, reserved characters could alter URL path interpretation or create inconsistencies with DFNS tooling. The reference SDK percent-encodes substituted path parameters using `encodeURIComponent`, establishing the expected URL-construction behavior. Encode `keyId` consistently in both the user-action challenge path and request URL so authentication and routing use the same canonical path.

```
typescript/packages/dfns/src/dfns-signer.ts
```

```

1  private async sendSignatureRequest(
2      request: GenerateSignatureRequest,
3      abortSignal?: AbortSignal,
4  ): Promise<SignatureBytes> {
```

```

5      // keyId is server-issued (from the wallet response) and this path is signed
      into the
6      // Dfns user-action challenge, which must match the routed request path
      verbatim – so it
7      // is interpolated raw rather than percent-encoded.
8      const httpPath = `/keys/${this.keyId}/signatures`;
9      const requestBody = JSON.stringify(request);
10     [...]
11  }

```

Remediation

1. Require each decoded component to be exactly 32 bytes and reject any other length before concatenating them into the 64-byte signature.
2. Percent-encode `keyId` with `encodeURIComponent` and use the identical encoded path for both DFNS user-action signing and the HTTP request.

Patch

Resolved in [PR#314](#).

Pre-Send Failures Misclassified As Unconfirmed Broadcasts

ID	Severity	Status
OS-SKC-SUG-04	● Informational	✓ Resolved

Description

1. `http::providerMayHaveAccepted` treats every error without a trusted HTTP status as evidence that the provider may have accepted the operation. However, a pre-request cancellation and some `HTTP_ERROR` cases, such as connection setup failures, may occur before any request reaches the provider. These errors therefore return true despite acceptance being impossible, causing callers to treat a safely retryable operation as uncertain and require unnecessary reconciliation.

```
typescript/packages/core/src/http.ts
```

```

1  /**
2   * A 4xx other than 408 is the only create outcome that rules out a transaction;
3   * anything else (no response, timeout, 5xx, unusable success body) may already be
4   * executing. A 408 is a timeout reached while the request was being processed, so
5   * it does not rule the transaction out either.
6   */
7   export function providerMayHaveAccepted(error: unknown): boolean {
8       const status = providerStatus(error);
9       return status === undefined || status < 400 || status >= 500 || status ===
10      408;
11  }
```

2. In sign-and-send backends, `providerMayHaveAccepted` classifies even a pre-send cancellation as potentially accepted because the abort error has no HTTP status. The backend may consequently wrap the cancellation as `BROADCAST_UNCONFIRMED`, implying that the transaction could have been submitted. This is incorrect when cancellation occurred before transmission, because no provider-side operation exists to reconcile. Callers may therefore perform unnecessary status checks or delay a retry that is known to be safe.

Remediation

1. Track whether request transmission could have begun, and return false for failures conclusively detected before the request was sent.
2. Distinguish pre-send cancellation from ambiguous in-flight failures and propagate it directly instead of converting it to `BROADCAST_UNCONFIRMED`.

Patch

Resolved in [PR#314](#) and [PR#324](#).

Non-Broadcast Failures Misclassified as Broadcast-Unconfirmed

ID	Severity	Status
OS-SKC-SUG-05	● Informational	✓ Resolved

Description

1. `createTransaction` in Crossmint passes every `doRequest` error to `UnconfirmedUnlessRejected`, including serialization or request-construction failures that occur before any request reaches Crossmint. This can incorrectly return `BROADCAST_UNCONFIRMED` for outcomes known not to have been accepted. Only post-dispatch transport failures with uncertain delivery should receive that classification.

```
go/signers/crossmint/client.go
```

```

1 func (s *Signer) createTransaction(ctx context.Context, transaction,
  idempotencyKey string) (transactionResponse, error) {
2     [...]
3     status, body, err := s.doRequest(ctx, http.MethodPost, u, req, idempotencyKey)
4     if err != nil {
5         return transactionResponse{}, core.UnconfirmedUnlessRejected(status, "",
           idempotencyKey, err)
6     }
7     [...]
8 }
```

2. Fireblocks calls `UnconfirmedUnlessRejected` in `createTransaction` for `PROGRAM_CALL` creation failures even though these requests set `SignOnly: true` and never broadcast the Solana transaction. Consequently, failures may be mislabeled as `BROADCAST_UNCONFIRMED`, misleading callers into treating an uncertain signing request as a potentially submitted on-chain transaction.

```
go/signers/fireblocks/client.go
```

```

1 func (s *Signer) createTransaction(ctx context.Context, request
  createTransactionRequest, programCall bool) (createTransactionResponse, error) {
2     ambiguous := func(status int, respBody []byte, err error) error {
3         if !programCall {
4             return err
5         }
6         return core.UnconfirmedUnlessRejected(status, transactionIDFromBody(respBody),
           request.ExternalTxID, err)
7     }
8     [...]
9 }
```

Remediation

1. Return pre-dispatch errors directly and apply `UnconfirmedUnlessRejected` only after the request may have reached Crossmint.
2. Return a signing/request error for sign-only `PROGRAM_CALL` failures instead of classifying them as `BROADCAST_UNCONFIRMED`.

Patch

Resolved in [PR#314](#) and [PR#323](#).

Imprecise Wallet Availability Check

ID	Severity	Status
OS-SKC-SUG-06	● Informational	✓ Resolved

Description

`client::fetchWallet` accepts responses where `wallet.solanaDetails` is entirely absent, and `signer::IsAvailable` therefore returns true as long as the wallet request itself succeeds.

```
go/signers/utila/client.go
```

```
1 func (s *Signer) fetchWallet(ctx context.Context) (walletResponse, error) {
2     [...]
3     if wallet.Wallet == nil || (wallet.Wallet.SolanaDetails != nil &&
4         wallet.Wallet.SolanaDetails.Address == nil) {
5         return walletResponse{}, core.NewSignerError(core.CodeSerializationError,
6             "Failed to parse Utila fetch_wallet response")
7     }
8     return wallet, nil
9 }
```

This can report a non-Solana or incompletely configured wallet as available, even though initialization requires valid Solana details and such a wallet may be unable to sign transactions. This issue is also present in the Rust, TypeScript, and Python modules.

```
go/signers/utila/signer.go
```

```
1 func (s *Signer) IsAvailable(ctx context.Context) bool {
2     ctx, cancel := context.WithTimeout(ctx, core.AvailabilityTimeout)
3     defer cancel()
4     _, err := s.fetchWallet(ctx)
5     return err == nil
6 }
```

Remediation

Require `solanaDetails` and a valid address in `fetchWallet`.

Patch

Resolved in [PR#314](#).

Code Quality

ID	Severity	Status
OS-SKC-SUG-07	● Informational	✓ Resolved

Description

In `batch::stagger`, casting the dimensionless index to `time.Duration` before multiplication is numerically correct but semantically misleading, as it appears to multiply two time values. Writing `delay * time.Duration(index)` more clearly communicates that the delay interval is scaled by the task index.

go/core/batch.go

```

1 func stagger(ctx context.Context, start time.Time, index int, delay time.Duration)
   error {
2     [...]
3     wait := time.Until(start.Add(time.Duration(index) * delay))
4     if wait <= 0 {
5         return nil
6     }
7     [...]
8 }

```

Remediation

Rewrite the expression as `start.Add(delay * time.Duration(index))` to make the scalar multiplication semantically clearer.

Patch

Resolved in [PR#314](#).

Appendix A.

Vulnerability Rating Scale

We rated our findings according to the following scale. Vulnerabilities have immediate security implications. Informational findings may be found in the general findings.

Severity	Description
● Critical	Vulnerabilities that immediately result in a loss of availability, consensus divergence, or exploitable memory corruption with minimal preconditions.
● High	Vulnerabilities that may result in consensus divergence or denial of service but require specific transaction or network conditions to exploit.
● Medium	Vulnerabilities that may result in incorrect behavior or state divergence under specific configurations or edge cases.
● Low	Low probability vulnerabilities, which are still exploitable but require extenuating circumstances or undue risk.
● Informational	Best practices to mitigate future security risks. These are classified as general findings.

Resolution status is one of  Resolved,  Ack'd, or  To-do.

Appendix B.

Procedure

As part of our standard auditing procedure, we split our analysis into two main sections: design and implementation.

The design review asks whether the programs's architecture and operating assumptions support its intended behavior. We study the inputs it accepts, the decisions it makes, the information shared between components, and the services or participants it trusts. We then consider how it behaves when those inputs are unexpected or conflicting. The goal is to find conditions that can produce an incorrect decision, inconsistent state, or loss of service.

Design problems often arise from interactions rather than one defective function. Two components may each process a value correctly but interpret it differently. A failure may also leave one component ready to continue while another treats the same operation as incomplete. Local checks can look correct in both cases, so we follow the complete operation across component boundaries.

The implementation review asks whether the code matches the design. We trace untrusted data from its entry point to the point where it affects behavior, checking the assumptions made along the way. The review covers validation, error handling, state updates, concurrency, persistence, resource use, and cleanup. Its exact focus depends on the target's language, runtime, and architecture rather than a fixed list of bug classes.

The boundary between design and implementation is not exact. Missing validation may be a local coding error, while the same omission across several components may reflect an unclear rule. We classify a finding by its root cause, but judge its impact from the resulting behavior and the conditions needed to trigger it.

We begin by reading the target as a team and agreeing on its main components, trust boundaries, and expected behavior. Auditors then review different paths through the system and compare notes where those paths meet. We reproduce suspected issues with focused tests or traces when the setup allows. Before reporting a finding, another auditor reviews its trigger, impact, and proposed remediation.

While sometimes the line between design and implementation can be blurry, we hope this gives some insight into our auditing procedure and thought process